

## CSCE 606 SOFTWARE ENGINEERING · MODULE 11 · LECTURE NOTES

# Observability

*Knowing what your system is doing in production.*

## MODULE OUTCOMES

- Distinguish monitoring from observability.
- Use logs, metrics, and traces for their respective purposes.
- Design alerts that are worth waking someone for.
- Explain why percentiles matter and averages mislead.
- Explain SLOs and error budgets.

## 1 Monitoring and observability

|                  | Monitoring                                  | Observability                                                                                 |
|------------------|---------------------------------------------|-----------------------------------------------------------------------------------------------|
| Answers          | Questions you knew to ask.                  | Questions you did not anticipate.                                                             |
| Mechanism        | Pre-defined dashboards, thresholds, alerts. | Rich, high-cardinality event data that can be queried freely after the fact.                  |
| Typical question | 'Is CPU above 80%?'                         | 'Why are requests from customer 4471, on build 1.8.3, in eu-west, slow — but only for POSTs?' |
| Sufficient?      | Necessary, and not sufficient.              | What you need during an incident you have not seen before.                                    |

## DASHBOARDS CANNOT ANSWER NOVEL QUESTIONS

A dashboard displays what someone previously decided was worth displaying. A genuinely novel failure — which is what incidents mostly are — is by construction not on it.

The question that actually resolves an incident is nearly always new and specific: is this confined to one region, one version, one customer, one endpoint, one combination of all four?

Answering that requires **querying rich data** rather than reading pre-aggregated charts, and in particular it requires **high-cardinality** fields — user IDs, request IDs, build hashes — which most traditional metrics systems explicitly cannot store, because each distinct value creates a new time series.

**The test of an observability setup:** can you ask a question nobody anticipated, right now, without shipping code? If answering requires a deploy, you have monitoring.

## 2 The three signals

| Signal         | What it is                                                                    | Answers                                              | Cost profile                                                                        |
|----------------|-------------------------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Logs</b>    | Discrete timestamped events with context.                                     | 'What happened to <i>this</i> request?'              | High volume; expensive to store and index at scale.                                 |
| <b>Metrics</b> | Numeric aggregates over time windows.                                         | 'Is the error rate rising? Is latency up this week?' | Very cheap — fixed cost regardless of request volume. The right basis for alerting. |
| <b>Traces</b>  | The path of one request across every service it touched, with timing per hop. | 'Where did the 3 seconds go?'                        | Expensive per trace, so usually sampled.                                            |

The three are complementary rather than alternatives. Metrics tell you something is wrong and are cheap enough to evaluate continuously. Traces tell you which component is responsible. Logs tell you what that component was actually doing.

### 2.1 Structured logging

```
# Unstructured: readable by a human, not queryable
log.info(f"Order {order.id} failed for user {user.id}: {err}")

# Structured: queryable
log.info("order_failed", extra={
    "order_id": order.id,
    "user_id": user.id,
    "error_code": err.code,
    "amount": order.total,
    "trace_id": current_trace_id(),
})
```

With structured events, 'all order\_failed events with amount over 1000 in the last hour, grouped by error code' is a query rather than an exercise in regular expressions. The fields are available for aggregation, filtering, and grouping without anyone having anticipated the question.

#### THE TRACE ID IS THE CONNECTIVE TISSUE

Generate a unique identifier at the system's edge and propagate it through every service call, every queue message, and every log line.

Then a single identifier connects all three signals: a metric anomaly leads to a trace, the trace identifies the slow service, and the trace ID retrieves exactly the log lines from that request.

**Without propagated trace IDs, correlating signals is manual archaeology** — matching timestamps across systems with differently-skewed clocks, which is as unpleasant as it sounds.

This is the single highest-value piece of plumbing in observability, and it is far cheaper to add before you need it than during an incident.

**AVERAGES CONCEAL PRECISELY WHAT YOU NEED TO SEE**

A mean response time of 100 ms is consistent with every user experiencing 100 ms, and equally consistent with 99% of users experiencing 50 ms while 1% wait five seconds.

The second situation is a serious problem, and the average is incapable of distinguishing it from the first.

**Track percentiles: p50, p95, p99, and p99.9.** The tail is where real user pain lives — and your most engaged users encounter it most often, simply because they make the most requests. A 1% failure rate means a user making a hundred requests per session almost certainly hits it.

**Never average percentiles across instances or time windows.** The mean of two p99 values is not a p99 of anything and has no valid interpretation. Percentiles must be computed from the underlying distribution, which is why histogram-based metrics exist.

### 3 Alerting

Every alert is an interruption of someone's attention, and frequently of their sleep. The budget is small and should be spent deliberately.

#### ALERT ON SYMPTOMS, NOT CAUSES

'CPU above 80%' is a cause. It might indicate a problem, or it might indicate the system working hard and serving users perfectly well. Waking someone to tell them the computer is being used is not a good use of a person.

'p99 latency above 2 seconds for five minutes' is a symptom. Users are experiencing it now.

Alert on the four things users actually experience: **availability, latency, error rate, and throughput**.

Causes belong on dashboards, where they are consulted during diagnosis after a symptom alert has fired.

**The test:** if this alert fires and nobody needs to take action, it should not be an alert.

#### ALERT FATIGUE IS A RELIABILITY FAILURE

An engineer paged six times in a night learns, correctly, to acknowledge without reading. The sixth alert and the first receive identical attention: none.

**Then the genuine alert is missed too.** Noisy alerting does not merely waste time; it destroys the response capability it exists to provide, and it does so invisibly — the alerts are all being acknowledged.

This is structurally identical to the flaky test of Module 05 and the red build of Module 10. A signal that is usually noise is treated as always noise, and the system that depended on it quietly stops working.

**The rule: every page must be both actionable and urgent.** If action can wait until morning, it is a ticket. If no action is possible, it is a dashboard. If it fires regularly and is always ignored, delete it — deleting it loses nothing and recovers attention.

### 4 Service level objectives

| Term            | Definition                                | Example                                                                                                          |
|-----------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| SLI — indicator | The thing you measure.                    | Proportion of HTTP requests completing successfully in under 300 ms.                                             |
| SLO — objective | The target for that measurement.          | 99.9% over a rolling 30-day window.                                                                              |
| Error budget    | What the objective permits you to fail.   | 0.1% of requests — about 43 minutes of full unavailability per month, or a larger amount of partial degradation. |
| SLA — agreement | A contractual commitment, with penalties. | Set well below the SLO, so that missing the SLO is a warning rather than a breach.                               |

**THE ERROR BUDGET IS THE GENUINELY USEFUL IDEA**

An SLO on its own is a target. The **error budget** turns reliability into a resource that is spent deliberately.

**Budget remaining:** ship features, take risks, deploy more often. You have room.

**Budget exhausted:** feature work stops and reliability work starts, until the budget recovers.

This resolves the standard conflict between development, which wants to ship, and operations, which wants stability — not by argument but by a number both sides agreed to in advance. It converts a recurring political dispute into arithmetic, which is a remarkable thing for a metric to accomplish.

**100% IS THE WRONG TARGET**

Each additional nine of availability costs roughly an order of magnitude more than the last, and above a certain point users cannot perceive the difference — their own network connection is less reliable than your service, so your additional nines are invisible beneath their noise floor.

So perfect reliability is simultaneously unachievable and, past some point, worthless.

Choosing 99.9% *explicitly* means deciding that 43 minutes of unavailability per month is acceptable, and treating it as a budget to be spent rather than a failure to be avoided.

**An unspent error budget means you were too conservative** — you could have shipped faster and taken more risk. That reframing, that excess reliability is itself a cost, is the most counterintuitive and most useful idea in site reliability engineering.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                 | What it covers for this module                                                                                                                  |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Google — Site Reliability Engineering (free online)                      | SLOs, error budgets, alerting philosophy, and incident response. The standard reference, and free.                                              |
| Charity Majors et al. — Observability Engineering (sample chapters free) | The monitoring-versus-observability distinction and the argument for high-cardinality data.                                                     |
| OpenTelemetry documentation                                              | The vendor-neutral standard for traces, metrics, and logs. Instrument with this rather than a proprietary agent.                                |
| Gil Tene — How NOT to Measure Latency (free video)                       | Why averages mislead, why percentile averaging is invalid, and what coordinated omission does to your measurements. Essential and entertaining. |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Convert an application's logging from string formatting to structured events. Then answer a question with a query that would previously have required grep and manual counting.
- 2 Implement trace ID generation and propagation across at least two services. Demonstrate retrieving all log lines for one request.
- 3 Instrument a service with a latency histogram. Plot mean alongside p50, p95, and p99 under load, and construct a situation where the mean looks healthy and p99 does not.
- 4 Audit an existing alert configuration. Classify each alert as symptom or cause, and as actionable or not. Delete the ones that fail both tests.
- 5 Define an SLI and SLO for a service you run. Compute the error budget in minutes and compare against actual downtime over the last month.
- 6 Run a load test and use traces to find where the time goes in a slow request. Report the breakdown by service.
- 7 Deliberately introduce latency into one dependency and verify that your alerting notices it before you do.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish monitoring from observability, and say why dashboards cannot resolve novel incidents.
- 2 What question does each of logs, metrics, and traces answer best, and how do their costs differ?
- 3 Why does structured logging matter, and what role does the trace ID play?
- 4 Why do averages mislead, and why can percentiles not be averaged?
- 5 What is the difference between a symptom alert and a cause alert, and why does it matter?
- 6 Why is alert fatigue a reliability problem rather than merely an annoyance?
- 7 What is an error budget, and why is an unspent one a problem?